

```
; =====  
; PureBasic OOP - Reference Manual (English)  
; Official documentation for the transpiler and the OOP GUI framework  
; Author:      MicrodevWeb  
; =====
```

PureBasic OOP Reference Manual (English)

Welcome to the official documentation for the PureBasic Object-Oriented transpiler and language layer.

1. Foundations of Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a programming paradigm organized around **data** and their **associated operations**, p

It is built on 5 fundamental pillars:

1.1 Classes and Objects

- **Class**: The blueprint defining the structure (fields/attributes) and behavior (methods).
- **Object (Instance)**: A concrete entity instantiated in memory based on a class (e.g., class `Dog` instantiates the object `Buddy`)

1.2 Encapsulation

Encapsulation bundles data and the functions that manipulate them, restricting unauthorized external access:

- **Public**: Accessible anywhere (both inside and outside the object).
- **Protected**: Accessible only within the declaring class and its derived (child) subclasses.
- **Private**: Accessible strictly inside the declaring class.

1.3 Inheritance (`Extends`)

Inheritance enables a derived (child) class to reuse and extend fields and methods from a base (parent) class, promoting code reu

1.4 Polymorphism (Dynamic VTable Dispatch)

Polymorphism allows manipulating various derived objects uniformly via a reference to their common base class. Method calls dyn

1.5 Abstraction (Abstract Classes & Abstract Methods)

Abstraction defines a generalized contract without supplying all implementation details:

- **Abstract Class** (`Abstract Class`): An incomplete class serving as a blueprint/contract. It **cannot** be instantiated directly.
- **Abstract Method** (`Abstract Method`): A method prototype without a body. Any concrete child class **must** implement this m
- **Concrete / Default Method**: An abstract class can also provide methods with default implementation, which child classes can

2. PureBasic OOP Syntax & Grammar (.pbo)

2.1 Declaring Abstract and Concrete Classes

```
` ` `oop  
;  
; -----  
; 1. ABSTRACT CLASS (Base contract / blueprint)  
; -----  
Abstract Class Shape  
  Protected name.s  
  Protected color.s  
  
  ; Constructor  
  Public Method Init(name_p.s, color_p.s)  
  
  ; Abstract Methods (Contract: mandatory in concrete child classes)
```

```
Public Abstract Method.d CalculateArea()  
Public Abstract Method.d CalculatePerimeter()  
Public Abstract Method Draw()  
  
; Concrete Method with default implementation in abstract class  
Public Method DisplayInfo()  
  
; Destructor  
Public Method Free()  
EndClass
```

```
; -----  
; 2. CONCRETE CLASS (Inherits from Abstract Class)  
; -----
```

Class Rectangle Extends Shape

```
Protected width.d  
Protected height.d  
  
Public Method Init(name_p.s, color_p.s, w.d, h.d)  
  
; Mandatory implementations of abstract methods  
Public Method.d CalculateArea()  
Public Method.d CalculatePerimeter()  
Public Method Draw()  
  
; Overriding the default method  
Public Method DisplayInfo()  
  
Public Method Free()  
EndClass  
```
```

---

## 2.2 Constructor Overloading (`Init`) & Explicit Rule

In PureBasic OOP, a class can define **multiple overloaded constructors** to fit various use cases (from minimal default sizing for

```
```oop  
Class Person  
Protected name.s  
Protected age.i  
  
; Constructor 1: No parameter  
Public Method Init()  
This\name = "Anonymous"  
This\age = 0  
EndMethod  
  
; Constructor 2: Name only  
Public Method Init(name_p.s)  
This\name = name_p  
This\age = 0  
EndMethod  
  
; Constructor 3: Name and Age  
Public Method Init(name_p.s, age_p.i)  
This\name = name_p  
This\age = age_p  
EndMethod  
EndClass  
  
; Instantiation matching the desired constructor:  
Define *p1.Person = New Person()  
Define *p2.Person = New Person("Alice")  
Define *p3.Person = New Person("Bob", 30)
```

...

- > **Explicit Constructors Rule**:
- > Each class explicitly defines its own constructors. Subclasses do not implicitly inherit parent constructor signatures without declaration.

3. Responsive GUI Framework & Components (`src/ui/`)

The GUI framework provides responsive layout containers and standard controls, enabling modern application design without manual layout calculations.

3.1 UI Controls Constructor Reference

Component	Available Constructors (from simplest to most complete)	Description & Parameters
UI::Button	Init(text.s) Init(text.s, w.i, h.i) Init(x.i, y.i, w.i, h.i, text.s) Init(x.i, y.i, w.i, h.i, text.s, flags.i)	Standard button control.
UI::TextBox	Init() Init(defaultText.s) Init(defaultText.s, w.i, h.i) Init(x.i, y.i, w.i, h.i, defaultText.s) Init(x.i, y.i, w.i, h.i, defaultText.s, flags.i)	Single-line text input field.
UI::Label	Init(text.s) Init(text.s, w.i, h.i) Init(x.i, y.i, w.i, h.i, text.s) Init(x.i, y.i, w.i, h.i, text.s, flags.i)	Static text label.
UI::CheckBox	Init(text.s) Init(text.s, checked.b) Init(text.s, w.i, h.i, checked.b) Init(x.i, y.i, w.i, h.i, text.s, flags.i, checked.b)	Check/uncheck control.
UI::ComboBox	Init() Init(w.i, h.i) Init(x.i, y.i, w.i, h.i) Init(x.i, y.i, w.i, h.i, flags.i)	Dropdown select box.
UI::ProgressBar	Init() Init(min.i, max.i) Init(min.i, max.i, w.i, h.i) Init(x.i, y.i, w.i, h.i, min.i, max.i, flags.i)	Progress indicator.
UI::Slider	Init() Init(min.i, max.i) Init(min.i, max.i, w.i, h.i) Init(x.i, y.i, w.i, h.i, min.i, max.i, flags.i)	Value slider control.
UI::ListIcon	Init(title.s, colWidth.i) Init(title.s, colWidth.i, flags.i) Init(x.i, y.i, w.i, h.i, title.s, colWidth.i) Init(x.i, y.i, w.i, h.i, title.s, colWidth.i, flags.i)	Icon list control.
UI::Controls::ToggleSwitch	Init() Init(checked.b) Init(w.i, h.i, checked.b) Init(x.i, y.i, w.i, h.i, checked.b)	Toggle switch control.

3.2 Responsive Layout Panels

Layout Panel	Available Constructors	Behavior & Features
UI::Layouts::StackPanel	Init() Init(orientation.i) Init(orientation.i, spacing.i) Init(orientation.i, spacing.i, flags.i)	Stacks controls vertically or horizontally.
UI::Layouts::DockPanel	Init() Init(lastChildFill = #True) Init(lastChildFill.b, w.i, h.i)	Edge-docking layout.
UI::Layouts::Grid	Init() Init(w.i, h.i)	2D flexible grid supporting fixed pixels, "Auto" content sizing, and weighted stretching.
UI::Window	Init(title.s) Init(title.s, w.i, h.i) Init(title.s, w.i, h.i, flags.i) Init(title.s, x.i, y.i, w.i, h.i, flags.i, parent.i)	Main application window.

4. Multi-Window Responsive Example

```
oob
XIncludeFile "ui/UI.pbo"
Using UI
Using UI::Layouts

Class MainWindow Extends UI::Window {
    Protected *rootDock.DockPanel
    Protected *toolbar.StackPanel
    Protected *btnNew.Button
    Protected *table.ListIcon

    Public Method Init() {
        ; Centered, resizable 800x600 window
        Super::Init("Contact Manager", 800, 600)

        ; Root DockPanel
        This\*rootDock = New DockPanel(#True)
        This\SetRootComponent(This\*rootDock)

        ; Top Toolbar (Dock Top)
        This\*toolbar = New StackPanel(#UI_Orientation_Horizontal, 8)
```

```
    This\*btnNew = New Button("+ New Contact")
    This\*toolbar\AddChild(This\*btnNew)
    This\*rootDock\AddDockChild(This\*toolbar, #UI_Dock_Top)

    ; Central Table (Fills remaining area)
    This\*table = New ListIcon("Name", 200)
    This\*table\AddColumn(1, "Email", 250)
    This\*rootDock\AddDockChild(This\*table, #UI_Dock_Fill)
}

Define *app.UI::Application = New UI::Application()
Define *win.MainWindow = New MainWindow()
*app\Run()
```
```

---

## 5. Compilation & Build Guide

Transpile `.pbo` source to native `.pb`:

```
```cmd
"compiler/transpiler.exe" "src/main.pbo" "src/main_generated.pb"
```
```

Compile to Standalone Binary using PureBasic Compiler:

```
```cmd
"C:\Program Files\PureBasic\Compilers\pbcompiler.exe" "src/main_generated.pb" /EXE "src/app.exe" /THREAD /UNICODE /XP /U
```
```